PROGETTO RETI LOGICHE

1. Introduzione

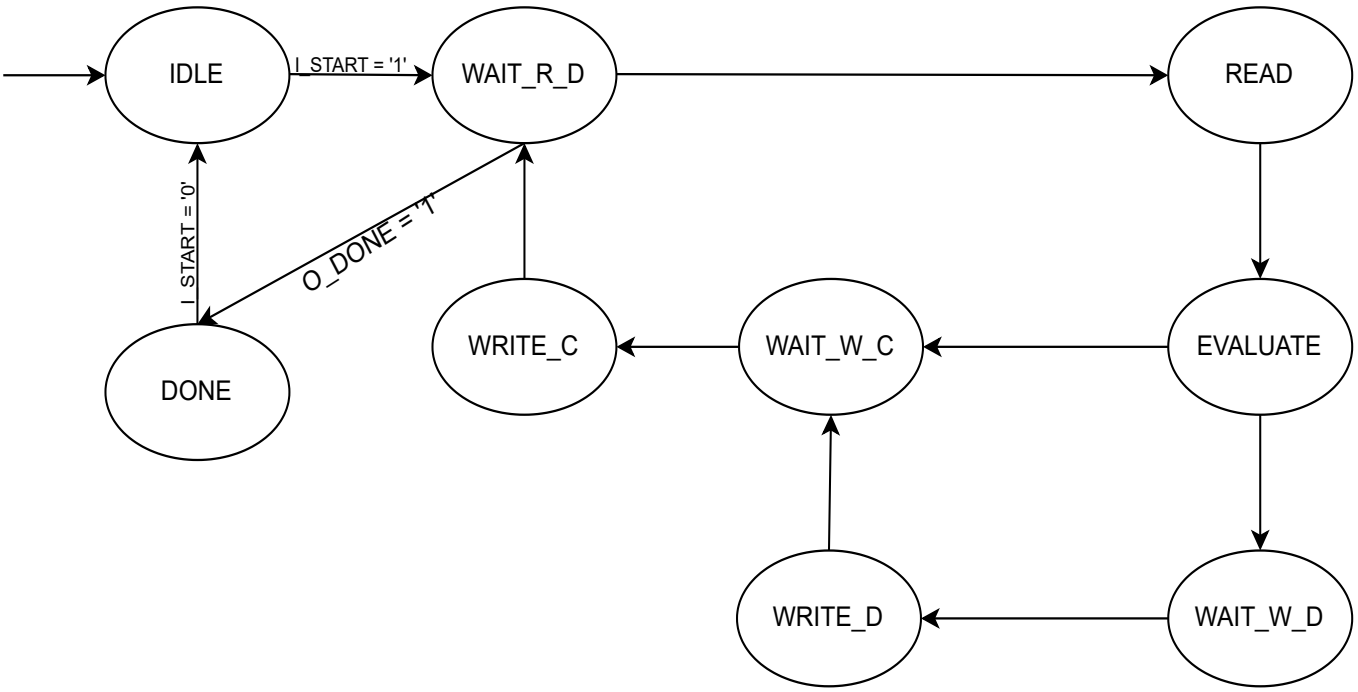
La specifica assegnata espone il seguente problema: è data una collezione di numeri interi positivi, compresi tra 0 e 255, in cui la cifra zero non indica un valore corretto, ma indica un valore nullo, il compito del modulo è quello di analizzare questa collezione e sostituire tutti gli zeri incontrati con l'ultimo valore valido letto. Ad ogni valore, inoltre, deve essere assegnato un valore di credibilità C. Questo valore può variare da 31, massima credibilità, a 0, minima credibilità. Il valore di credibilità è 31 se il numero è valido e viene ridotto di una unità, ogni volta che si incontra uno zero nell'intervallo tra due numeri validi.
Se la credibilità raggiunge il valore 0 allora smette di essere ridotto ulteriormente e rimane costante a 0 fino a che non si incontra nella collezione di interi un nuovo valore diverso da zero.
Per ogni numero nella collezione, il valore C viene memorizzato subito dopo il numero stesso in memoria. Quindi, se un numero occupa una posizione nella memoria all'indirizzo X, C occuperà la posizione immediatamente successiva all'indirizzo X+1.
In pratica il modulo deve riempire i "buchi" nella lista (dove ci sono gli zeri) con l'ultimo numero valido letto e tener traccia di quanto sia affidabile quel numero attraverso il valore C.

2. Architettura

L'architettura presentata è formata da una sola entity che corrisponde al modulo complessivo fornito dal docente con 5 segnali di input, un segnale di output per indicare il completamento del lavoro, 1 segnale di input verso la memoria e 4 segnali di output dalla memoria. Dai segnali di input il modulo riceve il primo indirizzo di memoria in cui trovare la collezione (i_add), il valore di 10 bit che indica quanti numeri si trovano all'interno della collezione (i_k), escludendo i valori di credibilità, e un segnale (i_start) che indica che il componente può cominciare l'elaborazione, oltre al segnale di clock e di reset.
Con i segnali di input e output della memoria il modulo è capace di leggere o scrivere all'interno di essa una parola per ciclo di clock, modificando la collezione dove necessario.
Il segnale di output indica che l'elaborazione è terminata e la sequenza con i giusti valori di credibilità e senza valori nulli, è stata correttamente salvata in memoria.
Nell' architecture del componente si è fatto uso di 6 Signal che ne facilitano la descrizione. L'architecture è composta da un solo processo che per ogni ciclo di clock verifica prima se il segnale di reset è 1, altrimenti segue il comportamento di una FSM.
La macchina a stati è implementata con 9 stati così descritti:

1. **IDLE**: stato iniziale (e di reset) in cui tutti i segnali vengono settati a un valore standard e al primo ciclo di clock in cui il segnale I_START = '1' procede con l'inizio dell'elaborazione passando allo stato successivo
2. **WAIT_R_D**: stato in cui si attende che la memoria esponga sul segnale I_MEM_DATA il valore richiesto dallo stato IDLE.

3. **READ**: stato in cui viene effettivamente letto dal componente il valore richiesto, inoltre si esegue un controllo per decidere se la sequenza data è stata completamente elaborata o meno. In caso affermativo si passa nello stato DONE asserendo il segnale o_done. In caso contrario si procede nello stato Evaluate.
4. **EVALUATE**: stato in cui si effettua un controllo sul valore della sequenza per verificare se esso sia nullo o meno. Nel primo caso si provvede a decrementare il valore di credibilità precedente, se necessario, e a sostituire all'interno della collezione il valore nullo con l'ultimo valore valido letto attraverso lo stato WAIT_W_D.
   Nel secondo caso, invece, si procede direttamente a scrivere il valore di credibilità '31' nella collezione in memoria attraverso lo stato WAIT_W_C.
5. **WAIT_W_D**: stato in cui si attende che la memoria scriva nella collezione il dato esposto sul segnale O_MEM_DATA all'indirizzo indicato dal segnale O_MEM_ADDR passando poi allo stato WRITE_D
6. **WRITE_D**: stato in cui dopo aver scritto il dato valido in memoria, si modificano i segnali per scrivere in memoria il valore di credibilità che corrisponde al dato appena scritto.
7. **WAIT_W_C**: stato in cui si attende la scrittura in memoria del valore di credibilità
8. **WRITE_C**: stato in cui è stato scritto il valore di credibilità. Si richiede alla memoria il prossimo dato della collezione e si ritorna allo stato WAIT_R_D.
9. **DONE**: stato finale in cui si arriva dopo aver completato l'elaborazione della collezione. Il segnale I_START è diventato 0 in quanto O_DONE è 1. Lo stato setta a 0 il segnale O_DONE, indica come prossimo stato lo stato IDLE e infine disabilita la memoria in quanto non necessaria fino a che non ci sarà una nuova sequenza da elaborare.



3. Risultati sperimentali:

Behavioral simulation

```
run all
Failure: Simulation Ended! TEST PASSATO (EXAMPLE)
Time: 2450 ns  Iteration: 1  Process: /project_tb/test_routine  File: /home/reti/Scrivania/project_tb.vhd
$finish called at time : 2450 ns : File "/home/reti/Scrivania/project_tb.vhd" Line 187
```

Il componente viene correttamente simulato in modalità behavioral in un
Analisi post-synthesis

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs*	77	0	0	134600	0.06
LUT as Logic	77	0	0	134600	0.06
LUT as Memory	0	0	0	46200	0.00
Slice Registers	67	0	0	269200	0.02
Register as Flip Flop	67	0	0	269200	0.02
Register as Latch	0	0	0	269200	0.00
F7 Muxes	0	0	0	67300	0.00
F8 Muxes	0	0	0	33650	0.00

```
run all
Failure: Simulation Ended! TEST PASSATO (EXAMPLE)
Time: 2453737 ps  Iteration: 1  Process: /project_tb/test_routine  File: /home/reti/Scrivania/project_tb.vhd
$finish called at time : 2453737 ps : File "/home/reti/Scrivania/project_tb.vhd" Line 187
```

Test

Le simulazioni superano il test bench base fornito come esempio dal professore.
Sono stati analizzati e testati i seguenti corner case:

1. Reset Asincrono: si resetta il componente in un momento qualsiasi dell'elaborazione e si verifica che il componente riesca comunque a elaborare la sequenza senza riscontrare difficoltà.
2. Sequenza che inizia con dato 0: il componente scrive come valore di credibilità 0 come richiesto da specifica
3. Sequenza più lunga di 31 numeri nulli: test che verifica la condizione che il valore di credibilità deve rimanere 0 nel caso in cui il numero di valori nulli dopo un numero valido sia più lungo di 30.
4. Due sequenze diverse nella stessa simulazione: il componente finita l'elaborazione della prima sequenza riesce a elaborarne una diversa anche in assenza di reset, come richiesto da specifica.

In tutti i test eseguiti si è assunto che i dati in memoria siano corretti.

4. Conclusioni

Si conclude che il componente descritto è sintetizzabile e supera correttamente tutti i test sopra specificati nelle tre simulazioni: Behavioral, Post-Synthesis Functional e Post_Synthetis Timing.

Si nota che il tempo di esecuzione è linearmente dipendente dal numero di interi presenti nella sequenza.
Inoltre grazie all'inserimento di due stati in più è possibile evitare, se il numero è valido, di fare una inutile sovrascrittura in memoria dello stesso e si procede direttamente a scrivere il valore di credibilità.